

Clase 20

Protocolos seguros

IIC2513 - Tecnologías y Aplicaciones Web

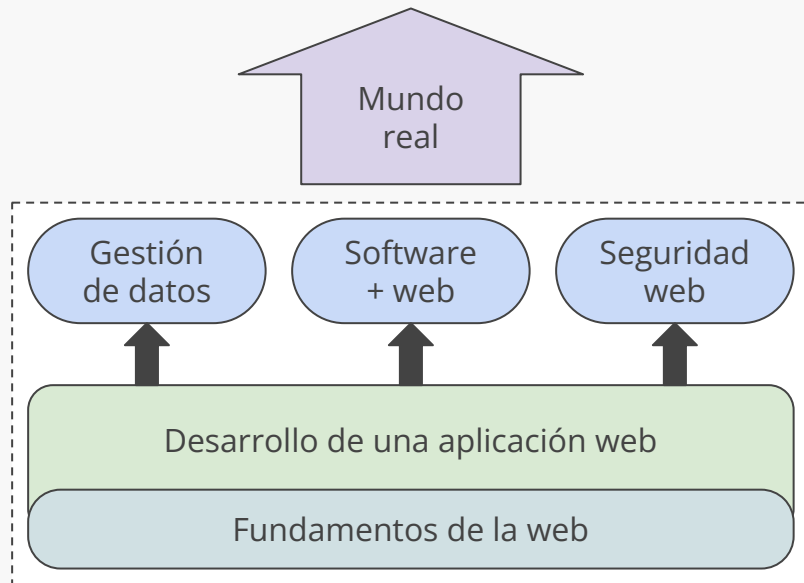
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. Día extra para la Actividad 6
2. Próxima semana: E2

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

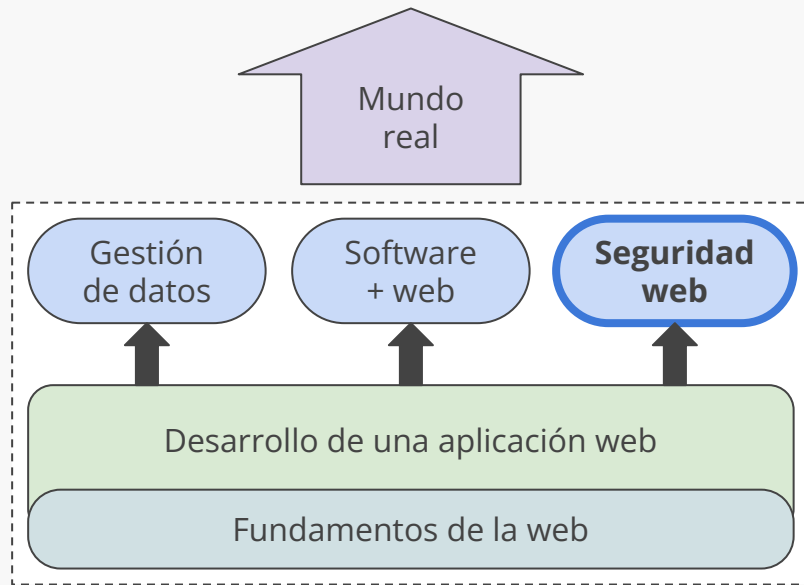


Contenidos del curso

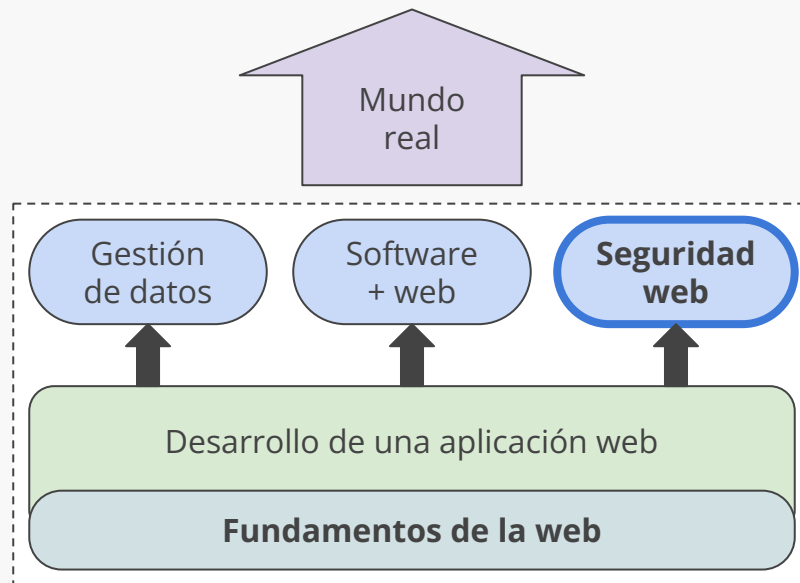
Seguridad web

Conoceremos los **ataques más comunes**, algunos **fundamentos de ciberseguridad** y los **protocolos** que permiten una experiencia segura...

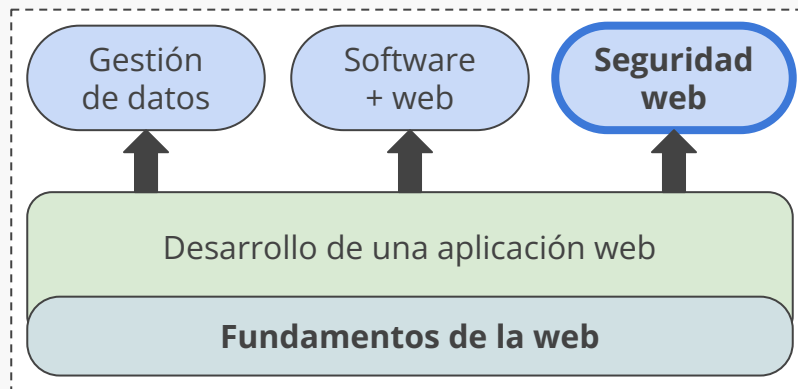
... para desarrollar **aplicaciones seguras** para los usuarios y los datos que estos generen



Contenidos del curso



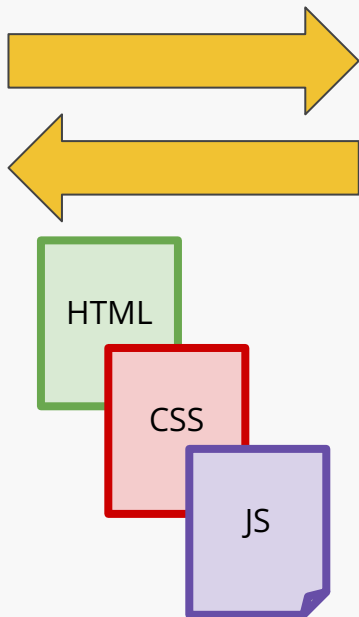
Contenidos del curso



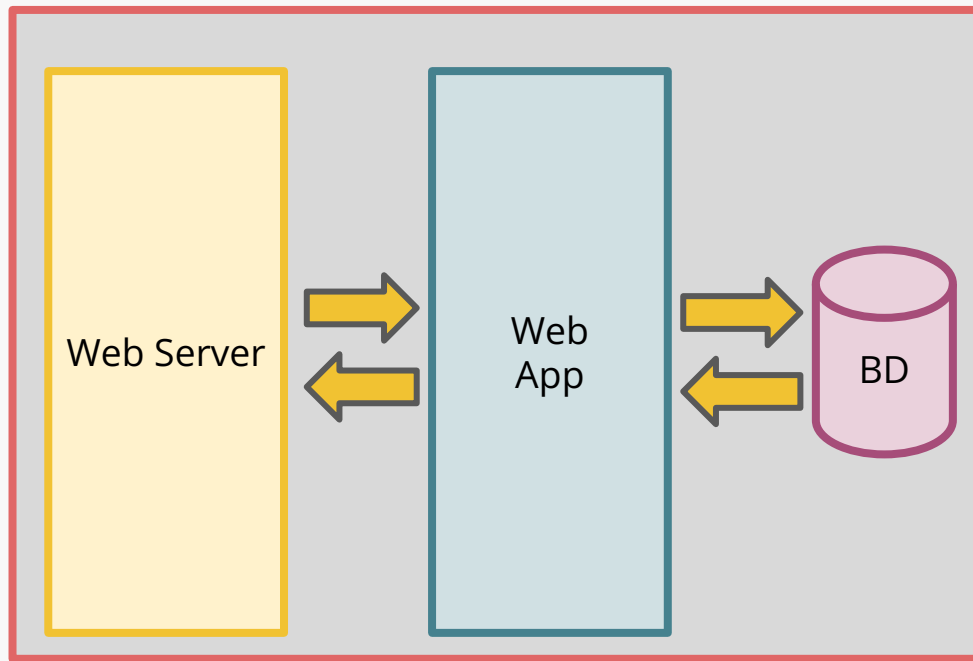
Menti#Hablemos sobre protocolos seguros

Aplicación web moderna

Cliente

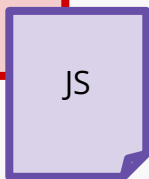
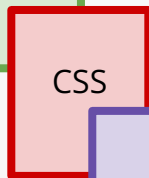
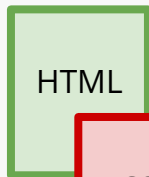


Servidor

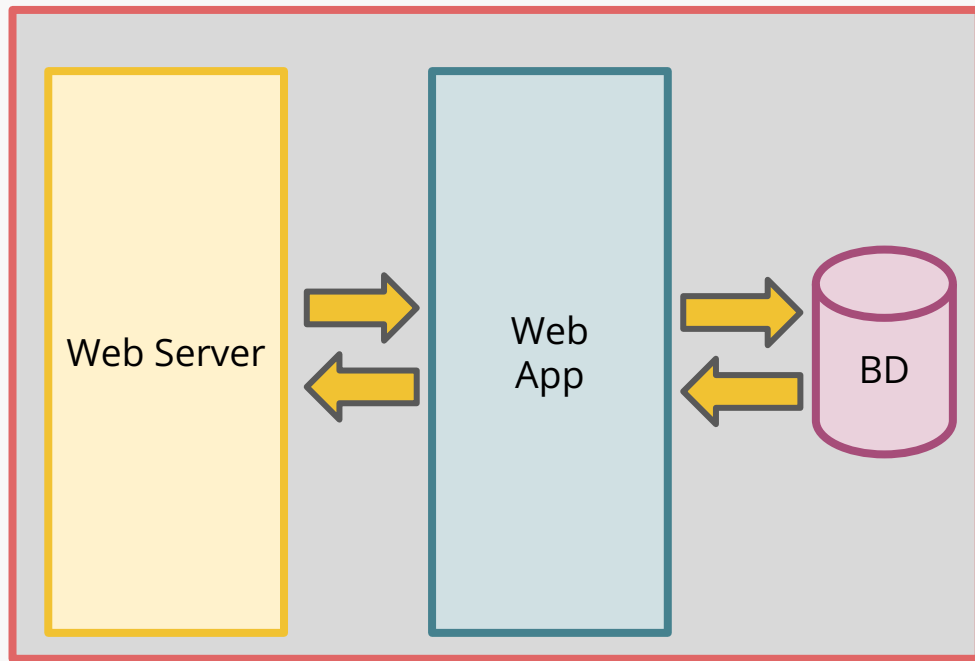


Aplicación web moderna

Cliente



Servidor



Aplicación web moderna

Cliente

Servidor



Agenda

🌟 HTTP

HTTPS

SOP

CORS

Versiones de HTTP

- **HTTP/0.9**: Primera versión. Solo método GET y solo HTML
- **HTTP/1.0**: Se incluye versión del protocolo, se incluyen headers para metadata y se soportan otros documentos además de HTML
- **HTTP/1.1**: Estándar aún en uso. Permitió reutilizar conexiones, responder en partes, alojar múltiples dominios en una IP (header Host)
- **HTTP/2**: Protocolo binario (no texto), soporta paralelismo, comprime headers, permite usar caché del cliente
- **HTTP/3**: Mejora en la codificación (por paquete) y velocidad, gracias al uso de QUIC en lugar de TCP.

Versiones de HTTP

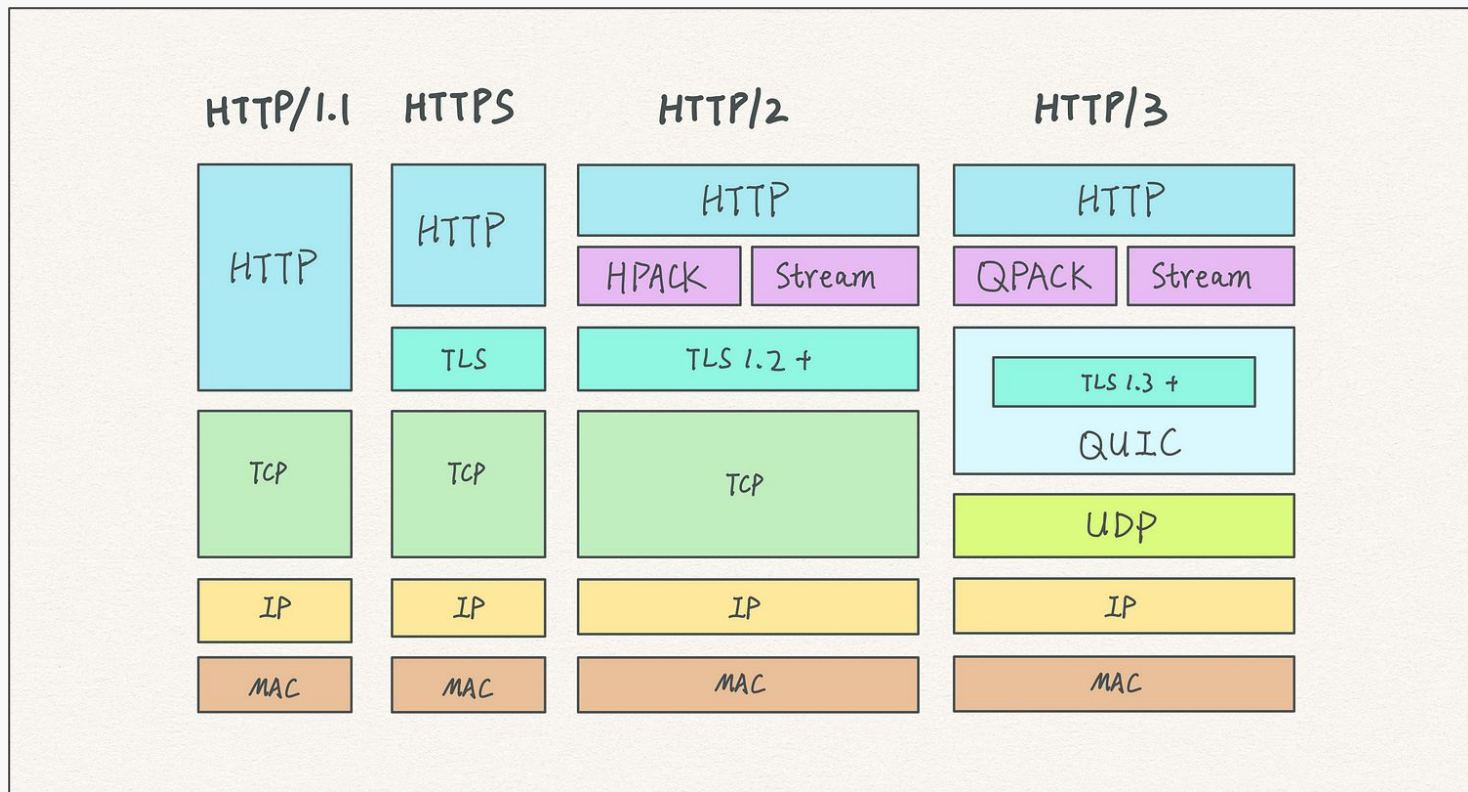
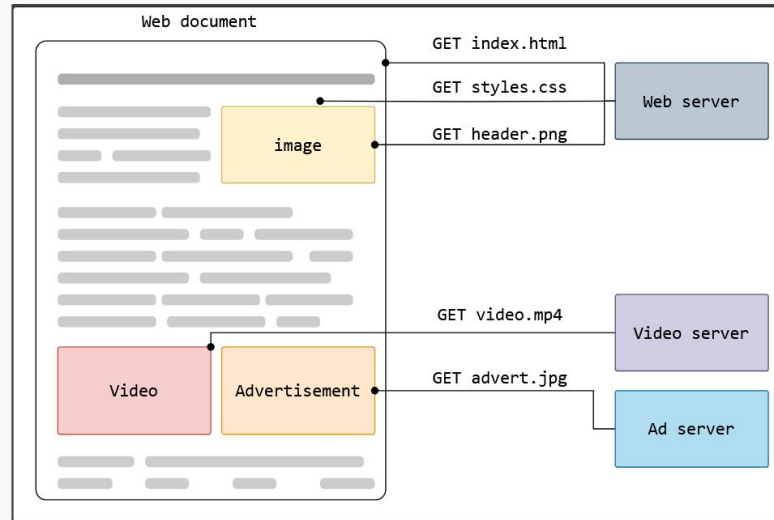


Figura: [HTTP/3, QUIC, and How it Works](#)

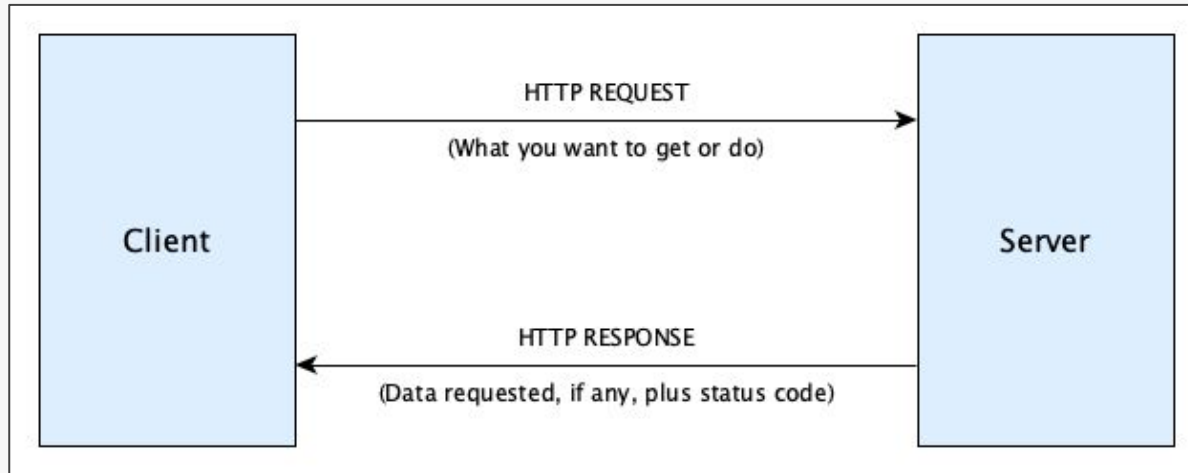
HTTP

HTTP (Hypertext Transfer Protocol) es el protocolo que permite **solicitar y utilizar datos y recursos**, como documentos HTML. Al estar basado en una estructura cliente-servidor, siempre quien solicita datos es quien los recibe (cliente, un navegador)



HTTP

HTTP (Hypertext Transfer Protocol) es el protocolo que permite **solicitar y utilizar datos y recursos**, como documentos HTML. Al estar basado en una estructura cliente-servidor, siempre quien solicita datos es quien los recibe (cliente, un navegador)



HTTP

¿Cómo se ve una *request* HTTP?

```
GET / HTTP/1.1 Host: developer.mozilla.org Accept-Language: fr
```

¿Cómo se ve una respuesta a la *request* HTTP?

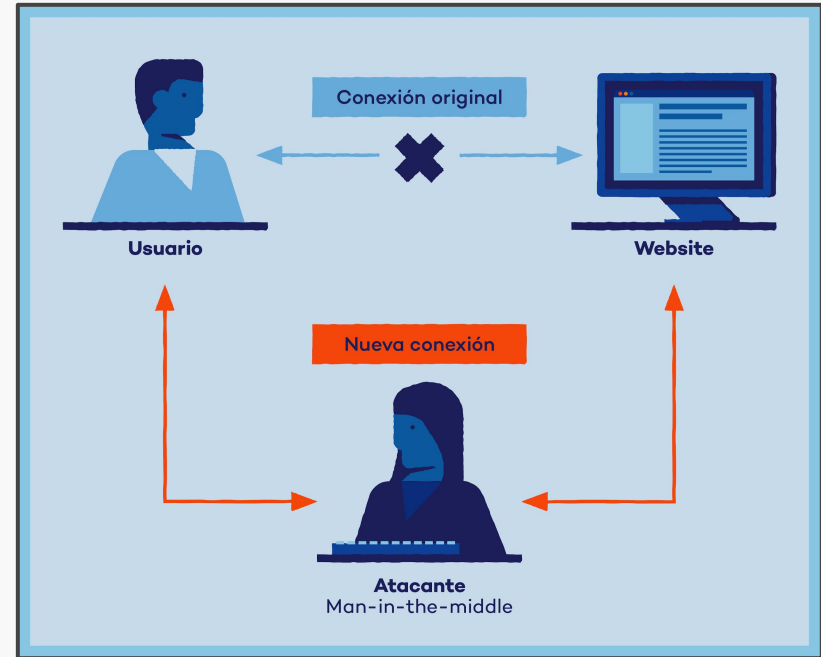
```
HTTP/1.1 200 OK  
Date: Sat, 09 Oct 2010 14:28:02 GMT  
Server: Apache  
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT  
ETag: "51142bc1-7449-479b075b2891b"  
Accept-Ranges: bytes  
Content-Length: 29769  
Content-Type: text/html
```

```
<!DOCTYPE html... (here comes the 29769 bytes of the requested web page)
```

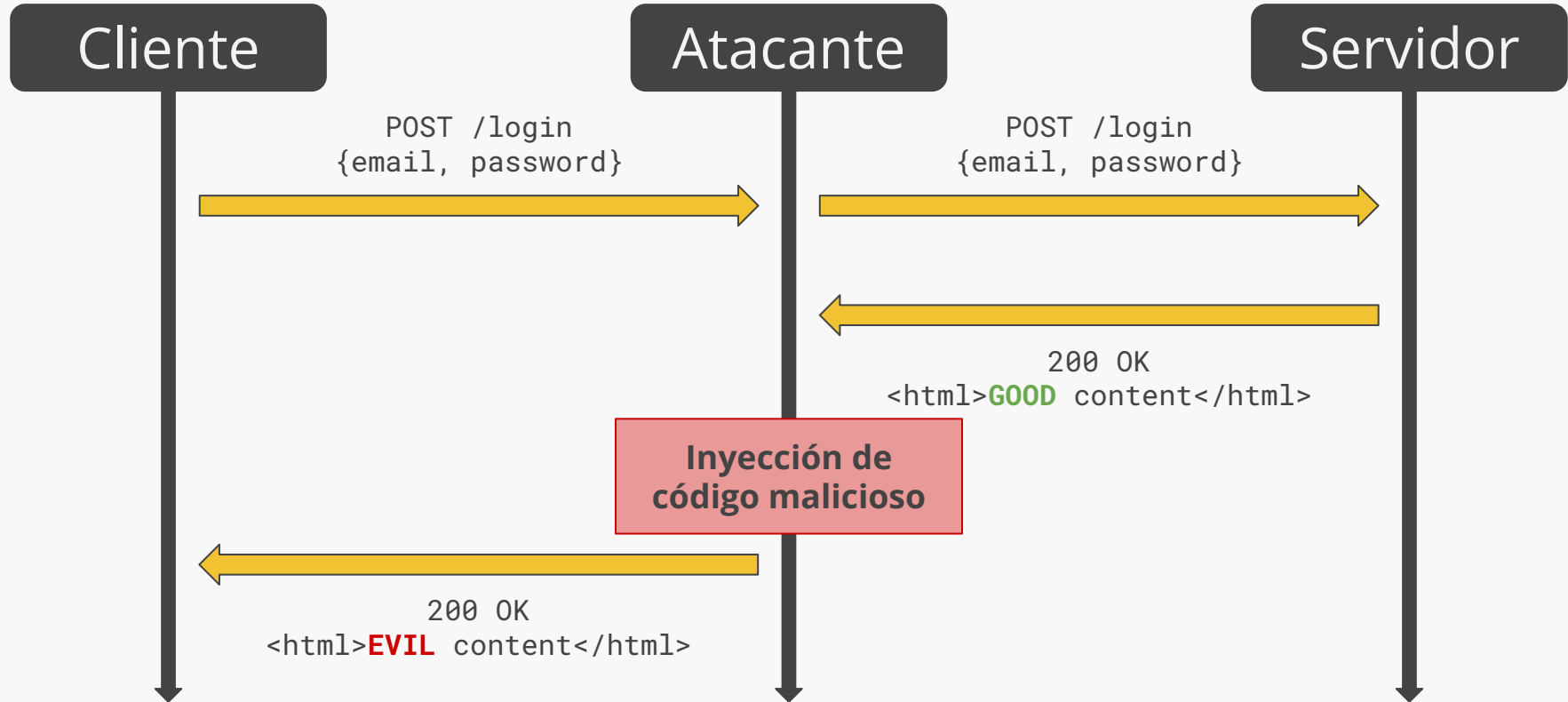

Man in the middle

Conexiones interceptadas por un tercero

- Solo “mirar”, inyectar código o robar información
- Suele darse redes wifi públicas



HTTP stateless



Agenda

HTTP

🌟 **HTTPS**

SOP

CORS

¿Qué es HTTPS?

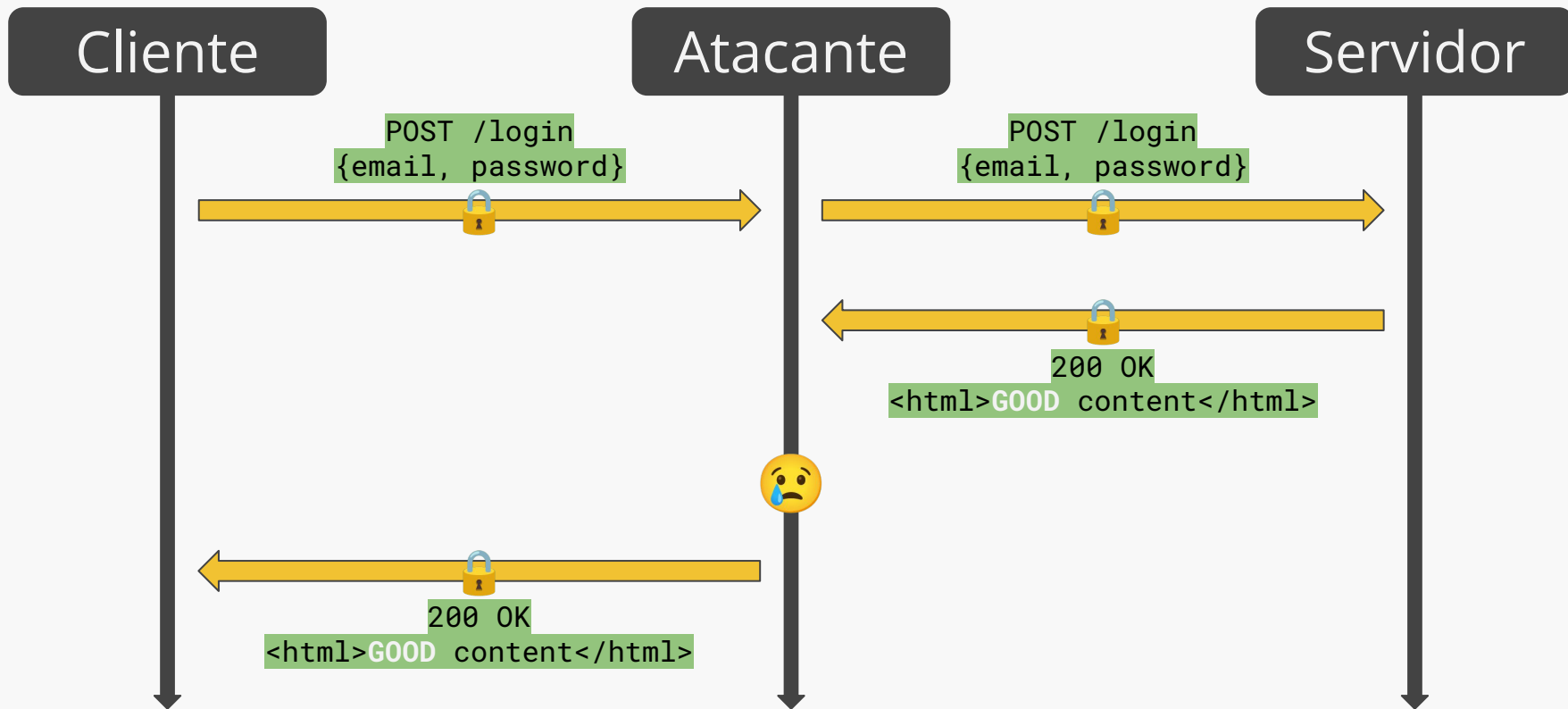
Versión segura de HTTP

HTTP sobre TLS

- **TLS** es un protocolo de seguridad que ofrece privacidad e integridad de datos para las comunicaciones en Internet
- **HTTPS** es la versión segura de HTTP, el principal protocolo para enviar datos en la web

Comunicación encriptada entre cliente y servidor

HTTP stateless

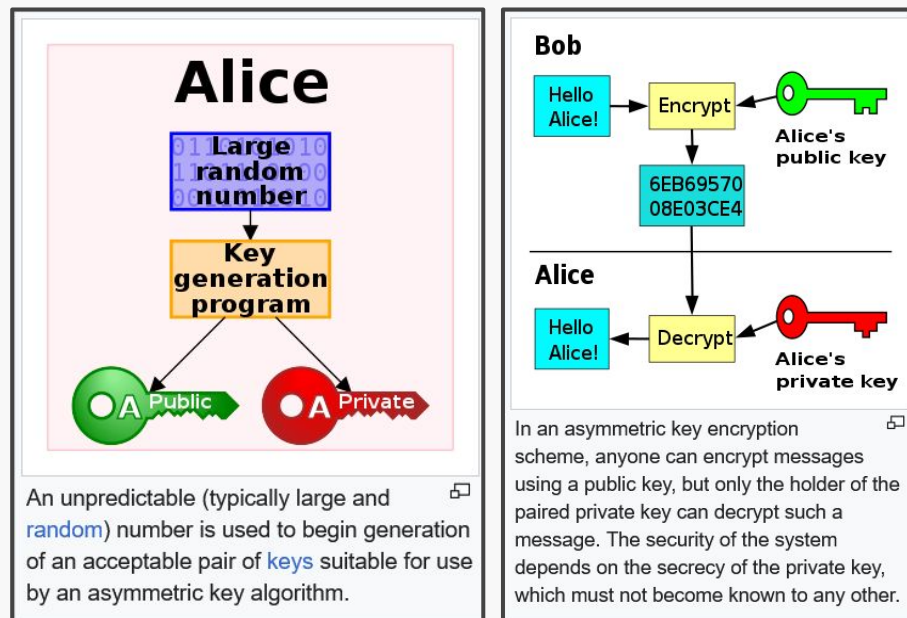


Public-key cryptography (encriptación asimétrica)

Utiliza un **par de llaves**: *private* (secreta) y *public* (compartida con el mundo)

[¿Cómo se generan estas llaves?](#)

Mensajes encriptados con *private key* sólo pueden ser descryptados con *public key* (y viceversa)



Public-key cryptography (encriptación asimétrica)

Utiliza un **par de llaves**: *private* (secreta) y

Pregunta#¿Cómo asegurar que una *public key* corresponde al servidor que espero?

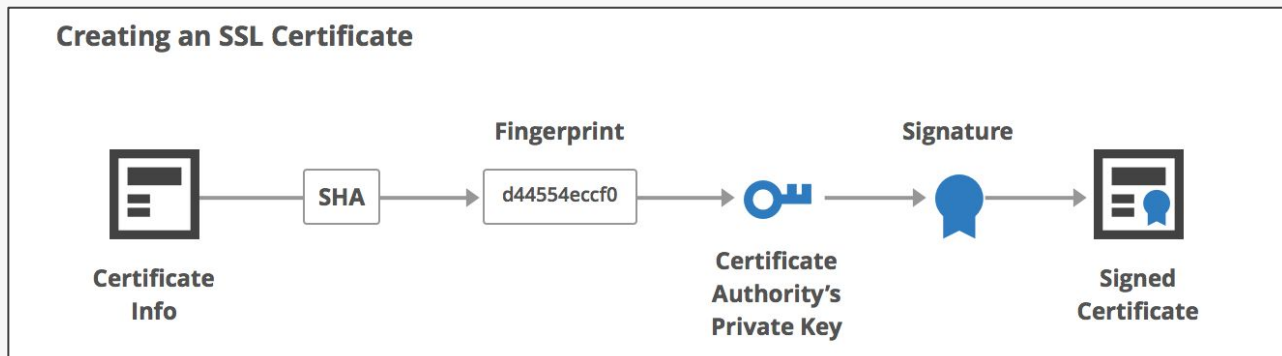
Mensajes encriptados con *private key* sólo pueden ser descryptados con *public key* (y viceversa)



Certificado TLS (SSL)

Servidor debe obtener un certificado (archivo) y guardarlo

- Entidad que entrega certificados: **Certificate Authority**
- Entidad certifica que **X dominio es dueño de Y public key**



Certificado TLS (SSL)

Luego los clientes pueden verificar que el certificado sea válido

- Navegadores recuerdan una lista de **Certificate Authorities** confiables
- Utilizan la *public key* de esos CAs para validar el certificado

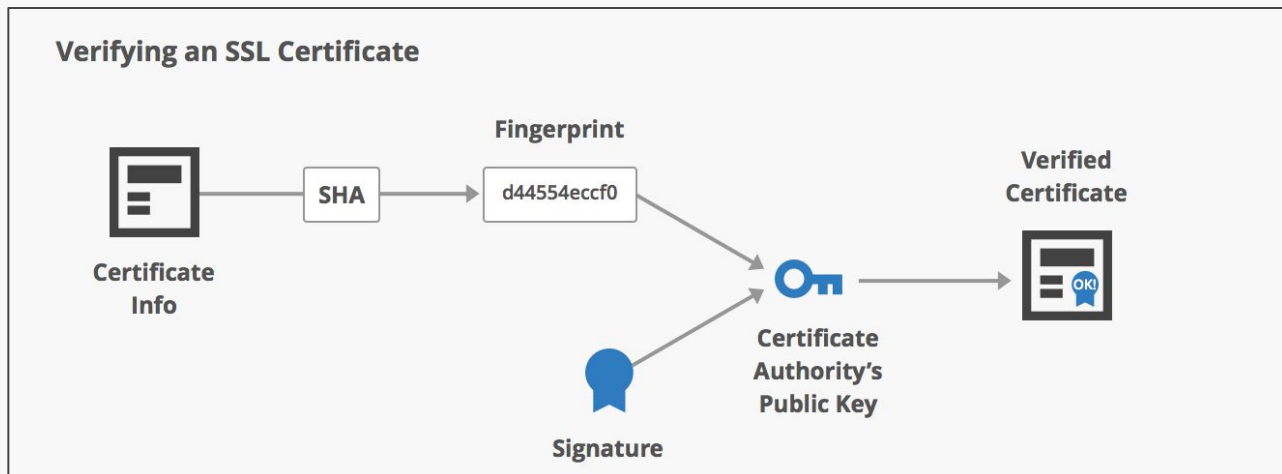


Figura: [TLS Certificate Optimization: The Technical Details behind "No Browser Left Behind"](#)

Flujo de comunicación con TLS

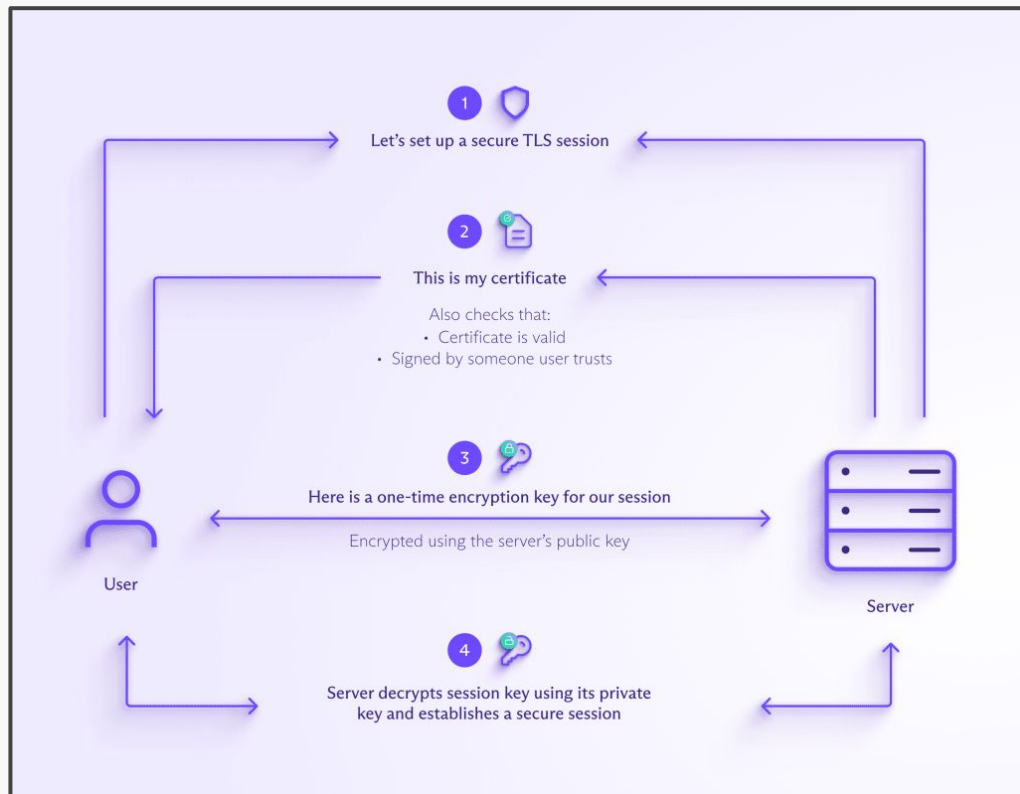


Figura: [What is a TLS/SSL certificate and how does it work? | Proton](#)

Flujo de comunicación con TLS



Flujo de comunicación con TLS

Cliente

El certificado solo se usa para
verificación de identidad del
servidor

Servidor

Pregunta# ¿Cómo se realiza el resto la comunicación entre cliente y servidor de forma segura?

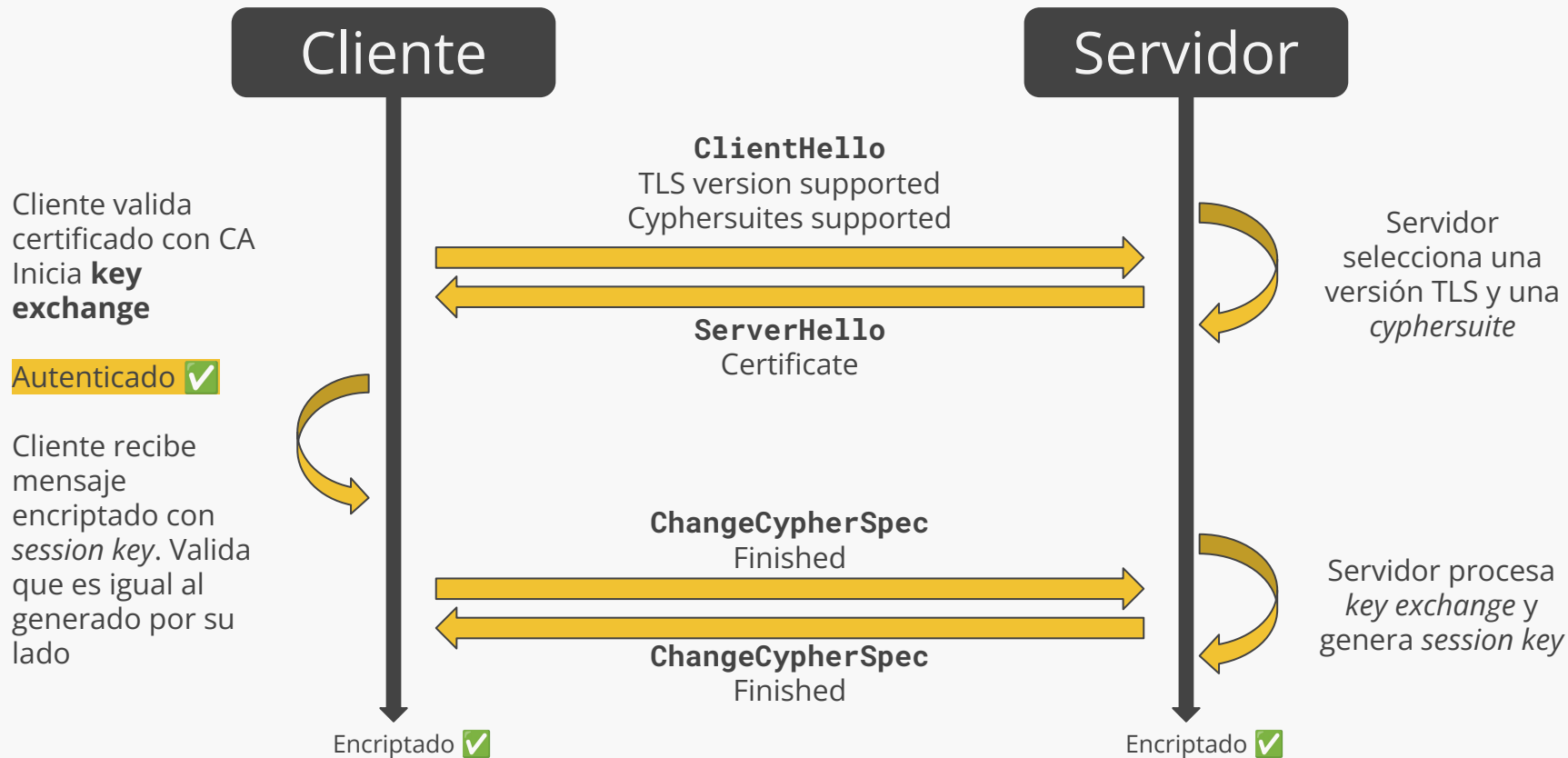
SESSION KEY

GET /profile
<encrypted>

SESSION KEY

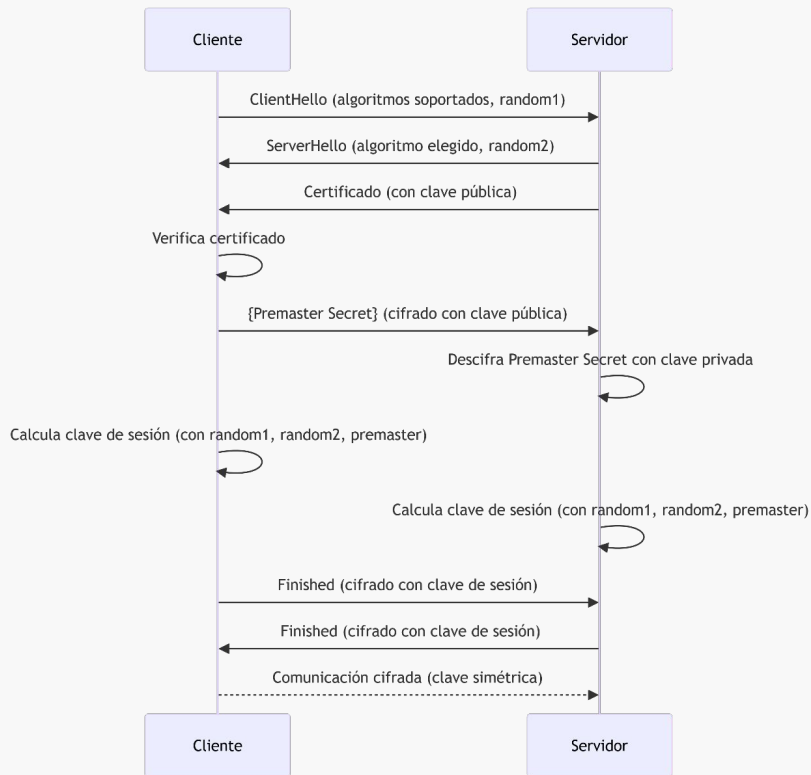
<encrypted>

Flujo de comunicación con TLS



Flujo de comunicación con TLS

- La clave pública no cifra todo el tráfico, solo el intercambio inicial. De hecho, ClientHello y ServerHello **vian sin cifrar**
- La clave pública del certificado se usa para establecer de manera segura una clave simétrica compartida: **clave de sesión**
- Toda la comunicación posterior se cifra usando esa clave simétrica
- Un MITM no podría romper esta seguridad porque nunca ve **premaster** ni la **clave privada del servidor**



https://aboutssl.org/ssl-tls-client-
authentication-how-does-it-work

Flujo de con

HTTPS uses asymmetric cryptography to encrypt the data. You have:

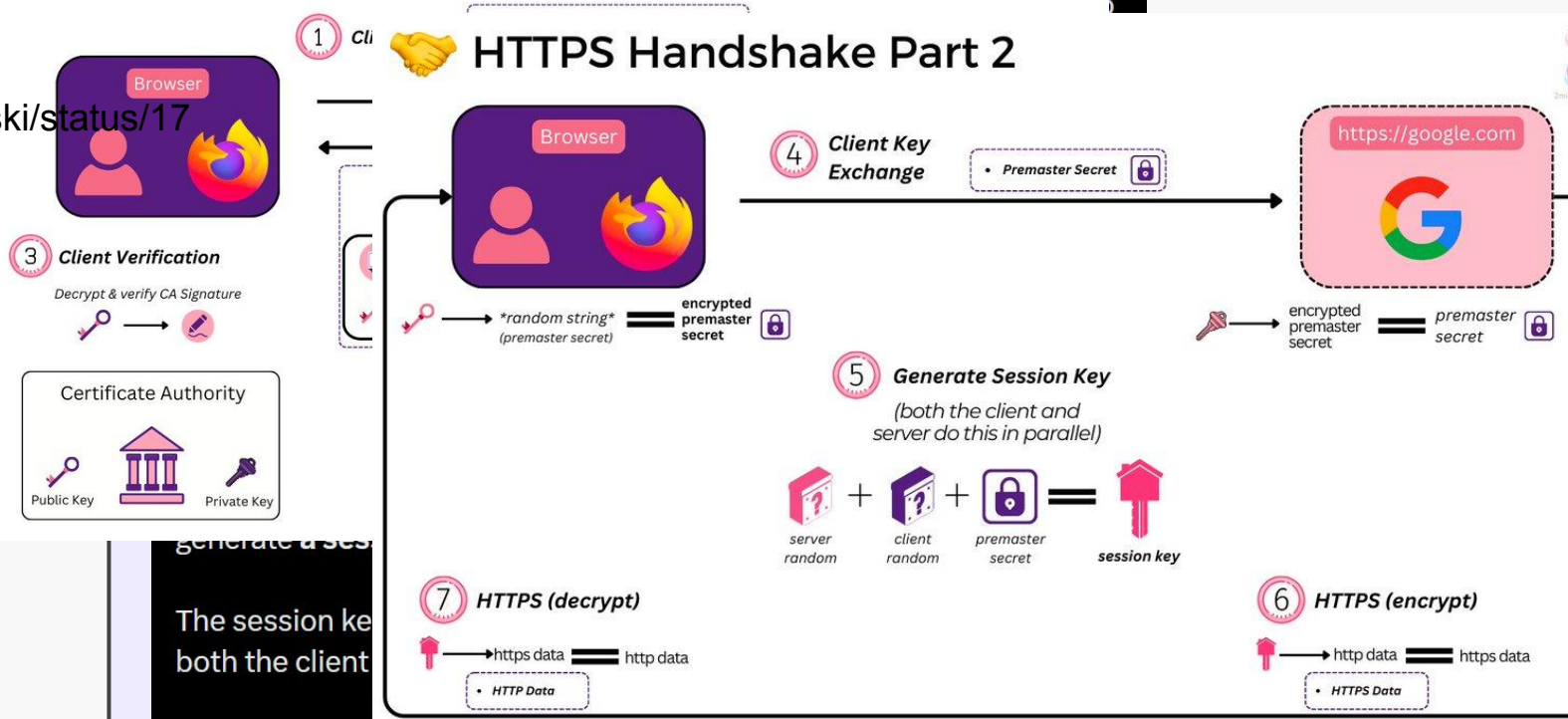
• a private key - the website's SSL certificate's secret key that isn't



HTTPS Handshake



https://x.com/BdKozlovski/status/17
047559628755415



The session key
both the client

Most interestingly, this key is deterministically generated by both the client and server independently - it never crosses the wire.

Figura: [What is a TLS/SSL certificate](#)

Beneficios HTTPS

¿Por qué deberíamos usarlo?

- Imagen de **confianza** hacia usuarios
- Navegadores modernos destacan sitios sin HTTPS como **no seguros**
- **HTTPS es más seguro**, tanto para clientes como para servidores
- Permite asegurar que **un servidor es quien dice ser**
- Capacidad de utilizar **features modernas de browsers** (geolocalización, *push*, etc)

HTTPS en la práctica

Cloudflare

Certificados TLS gratuitos

Let's Encrypt

Certificados TLS gratuitos

HTTPS is Easy!

Guía para incluir HTTPS

Agenda

HTTP

HTTPS

✨ **SOP**

CORS

Same-origin policy (Política del mismo origen)

La Same-Origin Policy es un **modelo de seguridad** fundamental implementado por los navegadores web que restringe cómo un documento o script cargado desde un origen puede interactuar con recursos de otro origen diferente

Un **origen** se define por la combinación de:

- **Protocolo** (http, https)
- **Host** (dominio o dirección IP)
- **Puerto** (explícito o implícito)

Origen: protocol + host + port

Origen: https://example.org:8080

http://example.org:8080

https://example.org

https://sub.example.org:8080

https://example.org:8080/page

https://example.com:8080

Same-origin policy (Política del mismo origen)

La Same-Origin Policy es un **modelo de seguridad** fundamental implementado por los navegadores web que restringe cómo un documento o script cargado desde un origen puede interactuar con recursos de otro origen diferente

Un **origen** se define por la combinación de:

- **Protocolo** (http, https)
- **Host** (dominio o dirección IP)
- **Puerto** (explícito o implícito)

Origen: protocol + host + port

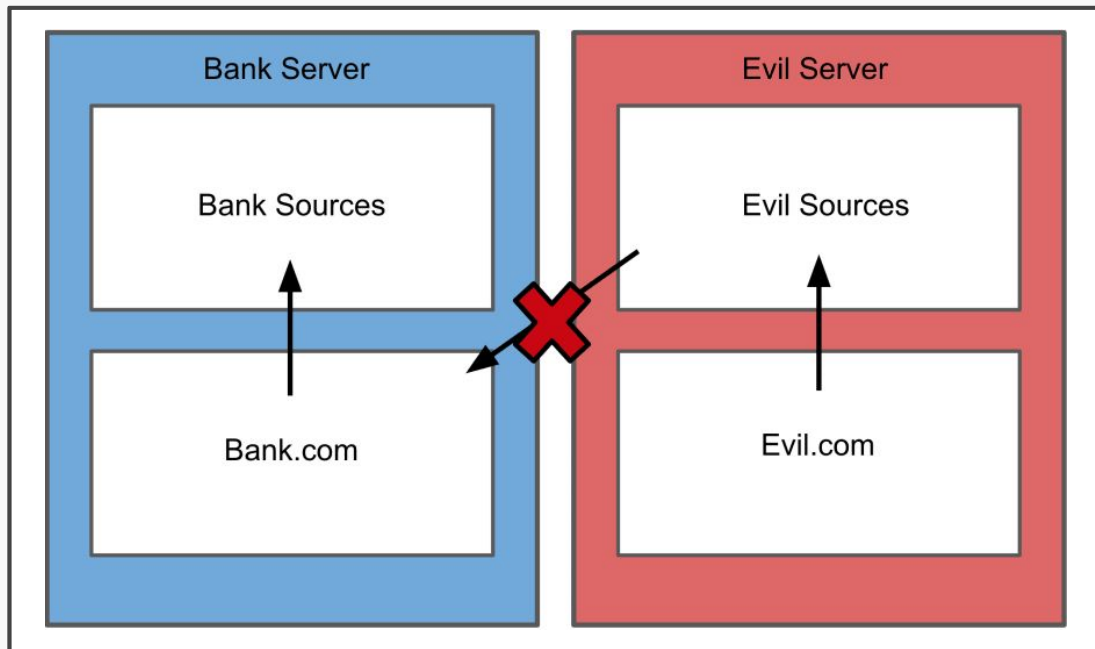
Origen: https://example.org:8080

- | | |
|------|-------------------------------|
| ✗ No | http://example.org:8080 |
| ✗ No | https://example.org |
| ✗ No | https://sub.example.org:8080 |
| ✓ Sí | https://example.org:8080/page |
| ✗ No | https://example.com:8080 |

Same-origin policy (SOP)

Excepciones a SOP:

- Imágenes
- Scripts
- Stylesheets
- Media (video, audio)



Agenda

HTTP

HTTPS

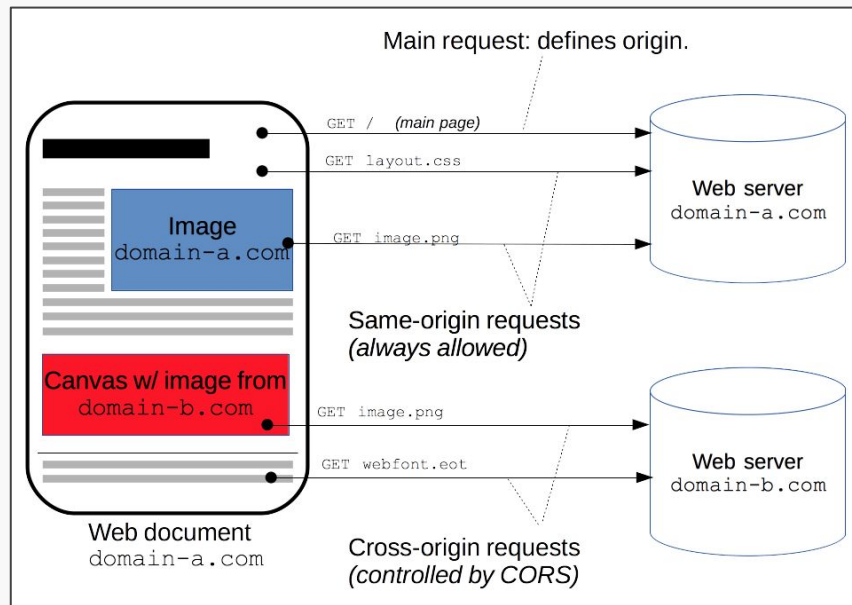
SOP

✨ **CORS**

¿Qué es *Cross-Origin Resource Sharing* (CORS)?

CORS es un mecanismo estándar del W3C que permite a los servidores web relajar las restricciones de la **Same-Origin Policy (SOP)** de manera controlada y segura, utilizando headers HTTP específicos

CORS no reemplaza SOP, sino que proporciona una forma segura de hacer excepciones controladas



CORS

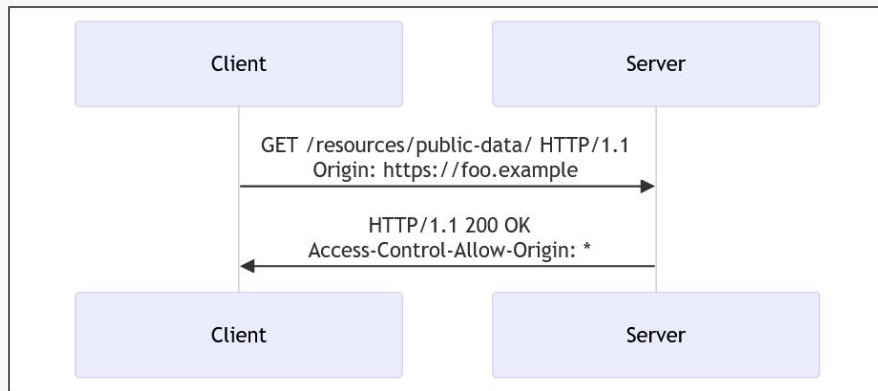
Tres escenarios posibles:

1. **Requests simples:** GET, POST (con ciertos content-types)
2. **Requests preflight:** los demás métodos que pueden tener *side-effects*
 - Involucra un request previo extra
3. **Requests con credenciales:** se pueden incluir credenciales de acceso

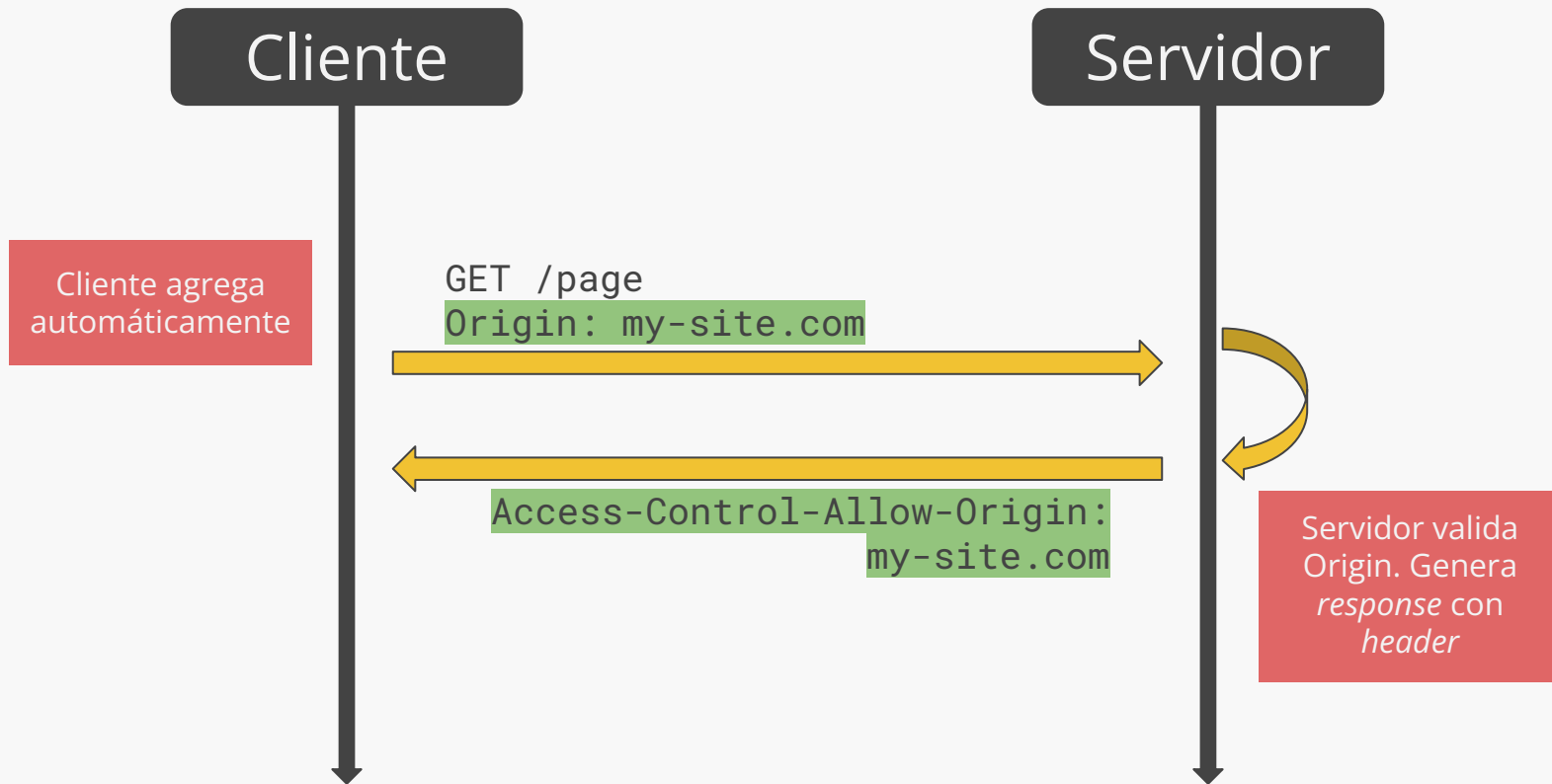
1) Request simple CORS

Request que cumple con las siguientes condiciones:

- **Métodos:** GET, POST, HEAD
- **Headers:** grupo reducido que contiene al menos:
 - Content-Type
 - Accept
- **Content-Type** sólo pueden tener los valores:
 - application/x-www-form-urlencoded
 - multipart/form-data
 - text/plain



1) Request simple CORS



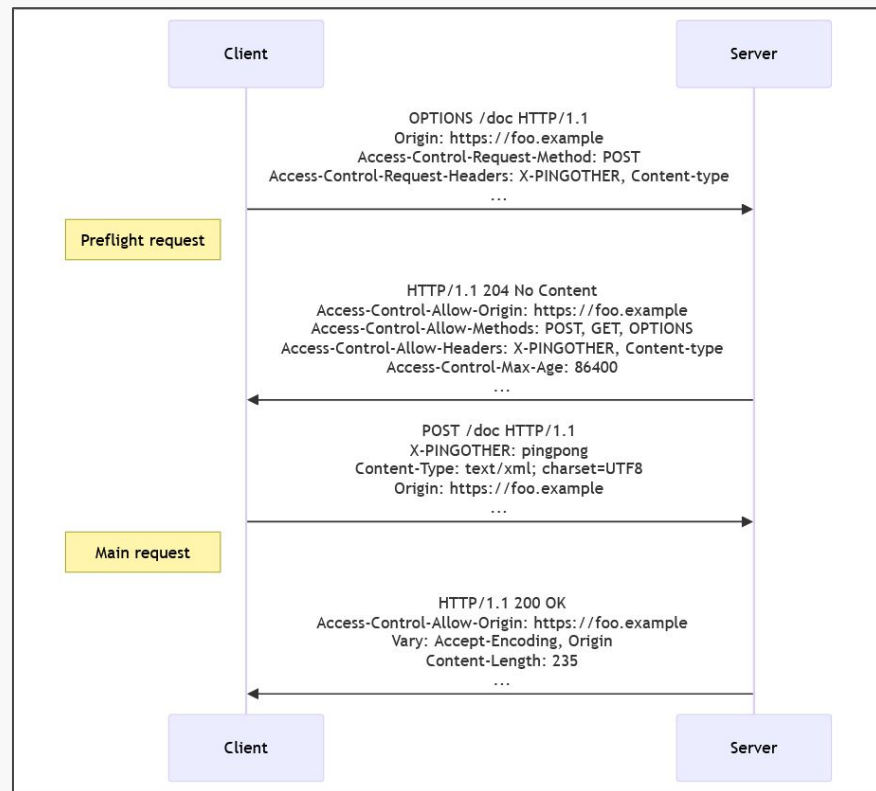
2) Request preflight CORS

Request que ocurre cuando **no se cumple** alguna **condición** de **request simple**. *Browser* envía primero un **request HTTP con método OPTIONS** para **determinar si es posible** enviar el request original

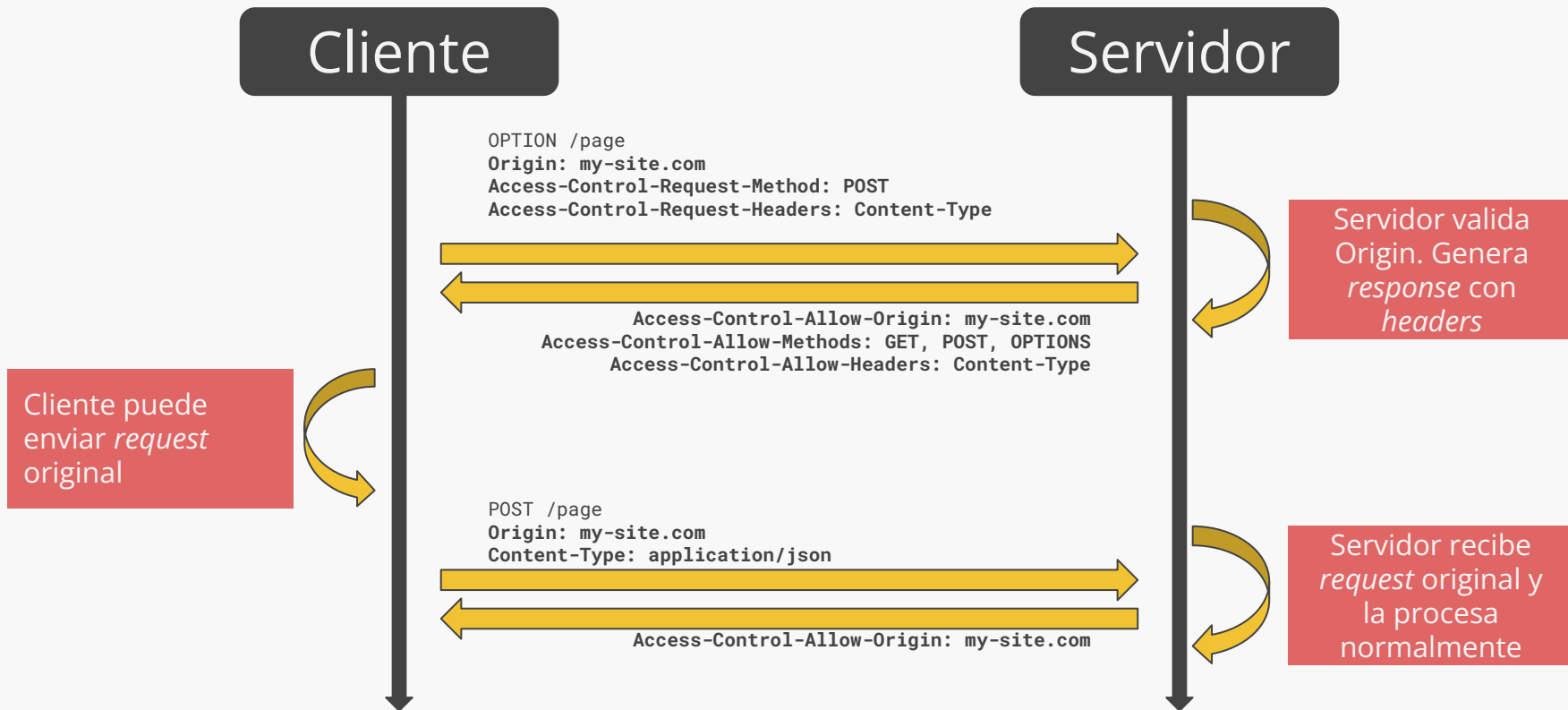
Request con **información en headers** del request original (indicando intención):

- Access-Control-Request-Method
- Access-Control-Request-Headers

Browser determina automáticamente qué valores enviar en headers basado en el request original



2) Request preflight CORS



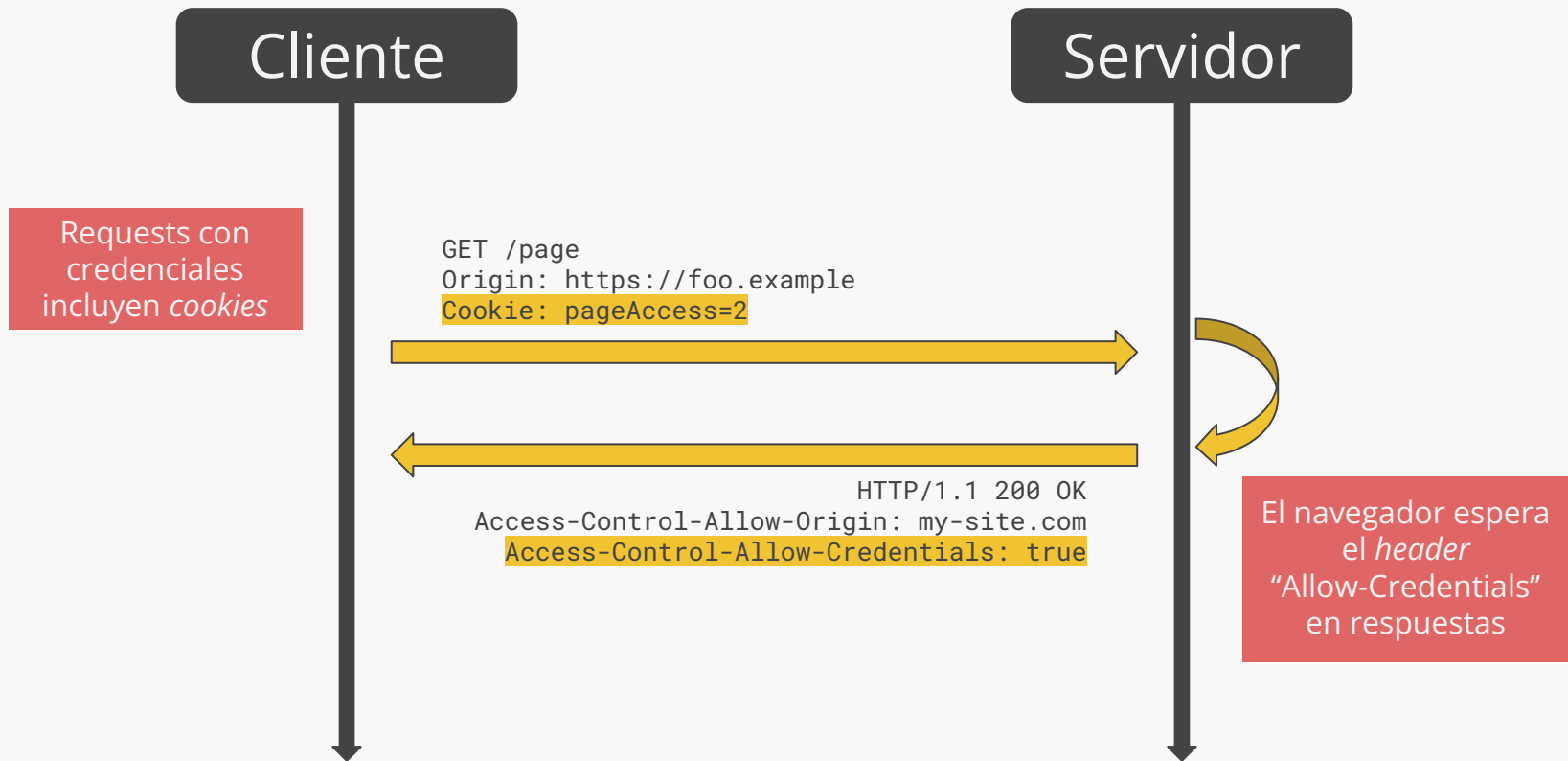
3) Request con credenciales

Los *requests with credentials* son solicitudes HTTP que incluyen credenciales como *cookies*, *headers* de autorización, o certificados de cliente TLS. Requieren manejo especial tanto del cliente como del servidor

1. **Browser detecta** que *request* incluye credenciales
2. **Verifica** si es un simple request o requiere preflight
3. **Si requiere preflight:**
 - Envía OPTIONS request (sin credenciales)
 - Servidor responde con headers permitidos
 - Browser valida que `Access-Control-Allow-Credentials: true` esté presente
4. **Si pasa validación:**
 - Envía request original con credenciales
 - Servidor procesa y responde con headers CORS apropiados



3) Request con credenciales



expressjs/cors - Node.js CORS middleware:

Librería de Express para configurar CORS

📖 README  MIT license



CORS

npm **v2.8.5** downloads **56M/month** ci **passing** coverage **100%**

CORS is a node.js package for providing a [Connect/Express](#) middleware that can be used to enable [CORS](#) with various options.

[Follow me \(@troygoode\) on Twitter!](#)

- [Installation](#)
- [Usage](#)
 - [Simple Usage](#)
 - [Enable CORS for a Single Route](#)
 - [Configuring CORS](#)
 - [Configuring CORS w/ Dynamic Origin](#)
 - [Enabling CORS Pre-Flight](#)
 - [Configuring CORS Asynchronously](#)
- [Configuration Options](#)
- [Demo](#)
- [License](#)
- [Author](#)

 opencollective.com/express

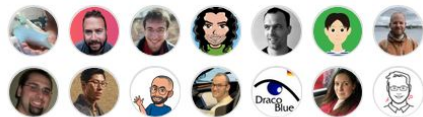
Packages

No packages published

Used by 8.2m



Contributors 51



[+ 37 contributors](#)

Languages

JavaScript 100.0%

koajs/cors - Cross-Origin Resource Sharing(CORS) for koa:

Librería de Koa para configurar CORS

[README](#) [Code of conduct](#) [License](#) [Security](#)

@koa/cors

npm v5.0.0 Node.js CI passing codecov 0% downloads 2.9M/month

[Cross-Origin Resource Sharing\(CORS\)](#) for koa

Installation

```
$ npm install @koa/cors --save
```

Quick start

Enable cors with default options:

- origin: * (v4 and before: the request's Origin header). This means that **by default the requests from all origin webpages will be allowed**. If you're running a generic API server, this is what you want, but otherwise you should look into changing the default to something more suitable to your application.
- allowMethods: GET,HEAD,PUT,POST,DELETE,PATCH

```
const Koa = require('koa');  
const cors = require('@koa/cors');
```

Used by 163k



Contributors 22



Languages

JavaScript 100.0%

Agenda

HTTP

HTTPS

SOP

CORS

✨ ✨ ¿Qué se viene pronto?

Clase 20

Protocolos seguros

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl